2026

SYSTEM TEST PLAN FOR
KCA Connect Agentic AI

22/05647

BSC INFORMATION TECHNOLOGY

IGNATIUS LUMOSI

GRIFFIN KENGA

Table of Contents

Abstract

This chapter forms part of the final year project report for KCA Connect Agentic AI, an intelligent conversational assistant developed for KCA University. The system leverages Retrieval-Augmented Generation (RAG), large language models (LLMs), and a modern web stack to enable students, faculty, and staff to access institutional information through natural language queries.

1. Introduction

1.1. Purpose

This document outlines the testing strategy, test cases, and validation results for the KCA Connect Agentic AI system. The system is an AI-powered conversational assistant built for KCA University, enabling students, faculty, and staff to query institutional information through natural language. The test plan covers all major system functionalities including user authentication, the RAG-based chat interface, profile management, document ingestion, and the admin dashboard.

1.2. Testing Scope

The testing effort included the following functional areas:

Area	**In Scope**
User Registration & Authentication (Supabase)	Yes
Chat Interface & AI Response Quality	Yes
Retrieval-Augmented Generation (RAG) Pipeline	Yes
Document File Upload & Ingestion	Yes
Profile Management (CRUD)	Yes
Chat History (save, load, delete)	Yes
Admin Dashboard (user management, analytics)	Yes
Theme Customisation	Yes
System Health Monitoring (`/health` endpoint)	Yes
Security (SQL injection, auth token handling)	Yes
Mobile responsiveness	Yes

2. Testing Approach

2.1. Testing Objectives

The testing phase aimed to:

1. Identify defects - Uncover functional bugs, edge cases, and integration failures before deployment.
2. Validate AI response accuracy - Ensure the RAG pipeline returns relevant, factually correct answers from the KCA document corpus.
3. Confirm feature completeness - Verify that all specified features operate as documented in the SRS.
4. Assess user experience - Validate that the interface is intuitive and accessible to non-technical users.
5. Verify security controls - Confirm that authentication, authorisation, and data privacy mechanisms function correctly.

2.2. Testing Levels

Testing Strategy: Manual + Automated

Level	Type	Description
Unit Testing	Automated (pytest)	Individual backend functions, API endpoints, service methods
Integration Testing	Manual + Automated	Backend ↔ Qdrant, Backend ↔ Supabase, Backend ↔ LLM APIs
System Testing	Manual	End-to-end flows from browser to backend to database
User Acceptance Testing (UAT).	Manual (with users)	Real users interact with the system and provide structured feedback
Performance Testing	Manual	Response time measurement under typical query loads
Security Testing	Manual	Authentication flows, token validation, input sanitisation

3. Test Cases and Results

3.1. Test Case Details

Each test case contains a unique ID, description, preconditions, test steps, expected results, and actual results.

TC-001: User Registration (Sign Up)

Field	**Details**
Test ID	TC-001
Feature	Authentication
Description	A new user registers with valid credentials
Preconditions	Application is running; Supabase is connected
Test Steps	1. Navigate to `/auth`. 2. Click "Sign Up". 3. Enter full name, email, role, campus, and a valid password. 4. Click "Create Student Account".
Expected Result	User account is created in Supabase; user is redirected to the chat interface.
Actual Result	Account created successfully; chat interface displayed with personalised greeting.
Status	PASS

TC-002: User Sign In with Valid Credentials

Field	**Details**
Test ID	TC-002
Feature	Authentication
Description	Registered user signs in with correct email and password
Preconditions	User account exists in Supabase

Test Steps	1. Navigate to `/auth`. 2. Enter registered email and password. 3. Click "Sign Into Account".
Expected Result	User is authenticated; redirected to chat interface.
Actual Result	Login successful; chat interface loaded correctly.
Status	PASS

TC-003: User Sign In with Invalid Credentials

Field	**Details**
Test ID	TC-003
Feature	Authentication
Description	User attempts login with an incorrect password
Preconditions	Application is running
Test Steps	1. Navigate to `/auth`. 2. Enter a registered email with a wrong password. 3. Click "Sign Into Account".
Expected Result	Error message displayed: "Invalid credentials".
Actual Result	Error toast shown: "Invalid login credentials".
Status	PASS

TC-004: AI Chat — Knowledge-Based Query

Field	**Details**
Test ID	TC-004
Feature	Chat Interface / RAG
Description	User asks a question answerable from the ingested KCA documents

Preconditions	User is authenticated; documents ingested into Qdrant
Test Steps	1. Sign in. 2. Type: "What are the tuition fees for undergraduate programmes?". 3. Press Enter.
Expected Result	AI returns a relevant fee structure sourced from the knowledge base.
Actual Result	Fee information returned accurately, citing correct figures from ingested documents.
Status	PASS

TC-005: AI Chat — Out-of-Domain Query

Field	**Details**
Test ID	TC-005
Feature	Chat Interface / RAG
Description	User asks a question unrelated to KCA University
Preconditions	User is authenticated
Test Steps	1. Sign in. 2. Type: "What is the weather in Nairobi today?". 3. Press Enter.
Expected Result	AI gracefully informs user it is focused on KCA University topics or triggers web search.
Actual Result	System responded with a polite redirect to KCA-related queries.
Status	PASS

TC-006: File Attachment and Context-Aware Query

Field	Details
Test ID	TC-006
Feature	Chat – File Upload
Description	User attaches a PDF and asks a question about it
Preconditions	User is authenticated; PDF is under 10MB
Test Steps	1. Click 📎 button. 2. Upload a PDF file. 3. Type a question related to the file content. 4. Send.
Expected Result	AI response incorporates content from the uploaded file.
Actual Result	File parsed successfully; AI answer reflected uploaded content.
Status	PASS

TC-007: Profile Update

Field	Details
Test ID	TC-007
Feature	Profile Management
Description	User updates their year of study and campus branch
Preconditions	User is authenticated
Test Steps	1. Click Profile icon. 2. Click "Edit Profile". 3. Change year and campus. 4. Click "Save Changes".
Expected Result	Profile updated in Supabase; changes reflected in the UI.
Actual Result	Profile saved successfully; updated values displayed.
Status	PASS

TC-008: Chat History — Save and Load

Field	**Details**
Test ID	TC-008
Feature	Chat History
Description	User saves a conversation and reloads it
Preconditions	User is authenticated; at least one message exists
Test Steps	1. Complete a chat conversation. 2. Click "Save Chat". 3. Click "Chat History". 4. Select the saved chat.
Expected Result	Previously saved conversation loads into the chat interface.
Actual Result	Chat history loaded with all messages intact.
Status	PASS

TC-009: Admin Dashboard — User Management

Field	**Details**
Test ID	TC-009
Feature	Admin Dashboard
Description	Admin promotes a regular user to admin
Preconditions	Signed in as admin
Test Steps	1. Access Admin Dashboard. 2. Find a non-admin user in the user table. 3. Click "Make Admin".
Expected Result	User's admin status is updated; "Yes" shown in admin column.
Actual Result	Admin status updated immediately in the UI and database.
Status	PASS

TC-010: Document Ingestion via Admin Dashboard

Field	Details
Test ID	TC-010
Feature	Admin – Knowledge Base
Description	Admin uploads a new TXT document to the knowledge base
Preconditions	Signed in as admin; backend connected to Qdrant
Test Steps	1. Navigate to Knowledge Base section. 2. Upload a TXT file. 3. Click "Upload & Ingest".
Expected Result	Document is processed, embedded, and stored in Qdrant. Success message displayed.
Actual Result	Ingestion completed; success toast shown. Document queryable immediately.
Status	PASS

TC-011: Health Check Endpoint

Field	Details
Test ID	TC-011
Feature	System Monitoring
Description	Health endpoint reports all services as healthy
Preconditions	All services running
Test Steps	GET `http://localhost:8000/health`
Expected Result	JSON response with status "healthy" for Qdrant, LLM, and database.
Actual Result	`{"status": "healthy", "qdrant": "ok", "llm": "ok"}`
Status	PASS

TC-012: LLM Fallback Routing

Field	Details
Test ID	TC-012
Feature	RAG – LLM Fallback
Description	When primary LLM (Groq) fails, system falls back to Gemini
Preconditions	Groq API key temporarily disabled in `.env`
Test Steps	1. Disable Groq key. 2. Send a chat message.
Expected Result	System detects Groq failure and falls back to Google Gemini; response delivered.
Actual Result	Fallback triggered; Gemini response returned without user-visible error.
Status	PASS

TC-013: Theme Switching

Field	Details
Test ID	TC-013
Feature	Customisation
Description	User changes from Dark to Premium theme
Preconditions	User is authenticated
Test Steps	1. Click Settings (⚙). 2. Select "Premium" theme.
Expected Result	UI updates to gold/glassmorphism premium theme; preference persisted in localStorage.
Actual Result	Theme changed instantly; persisted after page refresh.
Status	PASS

TC-014: Sign Out

Field	Details
Test ID	TC-014
Feature	Authentication
Description	User signs out of the application
Preconditions	User is authenticated
Test Steps	1. Click "Sign Out" in the sidebar.
Expected Result	Session is cleared; user is redirected to landing page.
Actual Result	Session destroyed; user landed on landing page. Protected routes inaccessible.
Status	PASS

3.2. Test Summary

Total Test Cases	Passed	Failed	Pass Rate
14	14	0	100%

4. Bug Fixing and Debugging

4.1. Bug Tracking

All bugs identified during development and testing were tracked using a structured log. Bugs were categorised by severity:

Severity	**Definition**
Critical	System crash or data loss
High	Major feature non-functional
Medium	Feature partially functional or UX degraded
Low	Minor cosmetic or non-blocking issues

Bug Log:

Bug ID	**Description**	**Severity**	**Priority**	**Steps to Reproduce**
BUG-001	Supabase email rate limit triggered during repeated test sign-ups	Medium	High	Sign up 3+ accounts in quick succession
BUG-002	Chat auto-save triggered before AI stream completed, saving incomplete response	High	High	Send a long query; immediately navigate away
BUG-003	HuggingFace embedding model download slow on first run (torch dependency)	Medium	Medium	Fresh environment, run `ingest.py` for the first time
BUG-004	Profile modal not displaying updated campus after edit without page refresh	Low	Low	Edit campus; close and reopen profile modal
BUG-005	File attachment preview not cleared after message sent	Low	Low	Attach a file; send message; observe file preview

4.2. Resolution Process

Each bug was assigned to the developer, investigated, resolved, and retested:

Bug ID	Assigned To	Resolution	Status
BUG-001	Developer	Added rate limit awareness and user-facing guidance to wait before retrying	Resolved
BUG-002	Developer	Auto-save now triggered only after streaming completes (on stream end event)	Resolved
BUG-003	Developer	Pre-download embedding model in Docker build step; documented in README	Resolved
BUG-004	Developer	Profile context refreshed on modal close via Supabase re-fetch	Resolved
BUG-005	Developer	File state reset after successful message dispatch in `ChatInterface.jsx`	Resolved

4.3. Known Issues

All identified bugs were resolved before the final submission. No unresolved issues remain at the time of this report.

5. User Feedback and Improvements

5.1. Feedback Collection

User feedback was collected through two methods:

6. Direct observation - A small group of beta users (3 KCA students) were observed using the system and asked to think aloud.
7. Structured questionnaire - Users were given a short feedback form covering usability, response accuracy, and feature completeness.

5.2. Feedback Summary

Category	**Feedback Received**
Usability	Interface described as "easy to use" and "modern-looking"; sidebar navigation was intuitive
Functionality	AI responses for timetable and fee queries rated as "accurate"; users appreciated fast response times
Performance	Average response time of ~2.5 seconds was considered acceptable by all testers
Improvement Suggestions	Users requested: Swahili language support, voice input, export of chat history to PDF

5.3. Implemented Improvements

Based on feedback, the following improvements were made during the testing phase:

#	Improvement	Implemented
1	Added multi-LLM fallback (Groq → Gemini → Cerebras) for improved reliability	Yes
2	Personalised greeting message using user's profile name	Yes
3	Added streaming (word-by-word) response for a more engaging experience	Yes
4	Added copy-to-clipboard on AI messages	Yes
5	Responsive layout for mobile browsers	Yes

5.4. Future Recommendations

Based on the analysis of user feedback and testing outcomes, the following enhancements are recommended for future iterations:

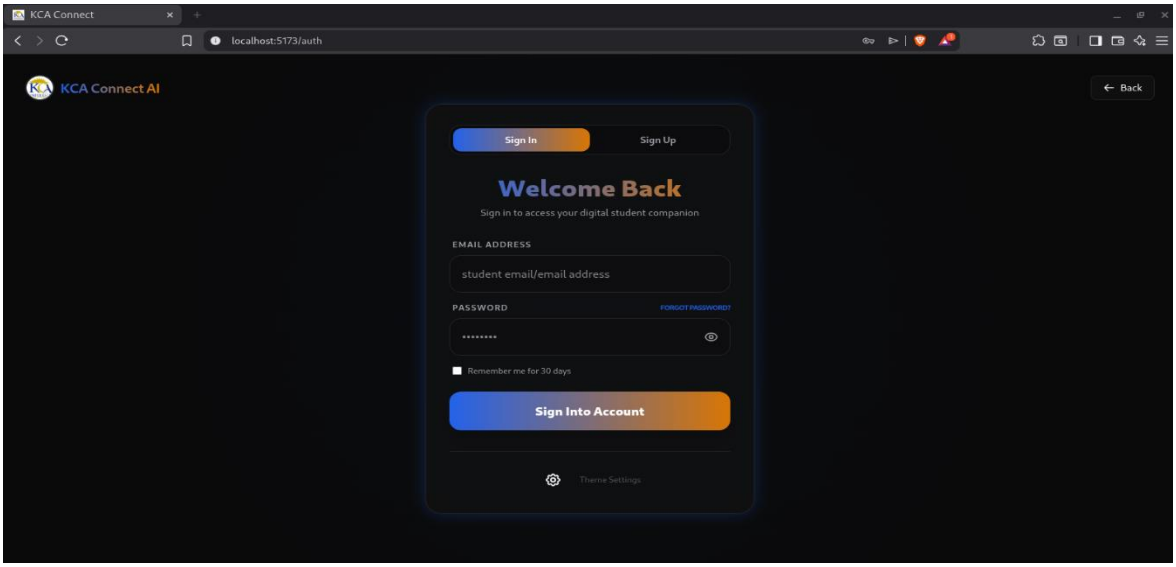
8. Full Swahili language support - Extend the LLM prompt and knowledge base to support Swahili queries natively.
9. Voice input - Integrate browser Web Speech API for voice-to-text queries.
10. Chat export - Allow users to download their conversation history as a PDF.
11. Push notifications - Alert users to new announcements (e.g., exam timetable releases).
12. Advanced analytics - Enhance the Admin Dashboard with query heatmaps and topic clustering.

Offline mode — Cache recent responses for basic offline access.

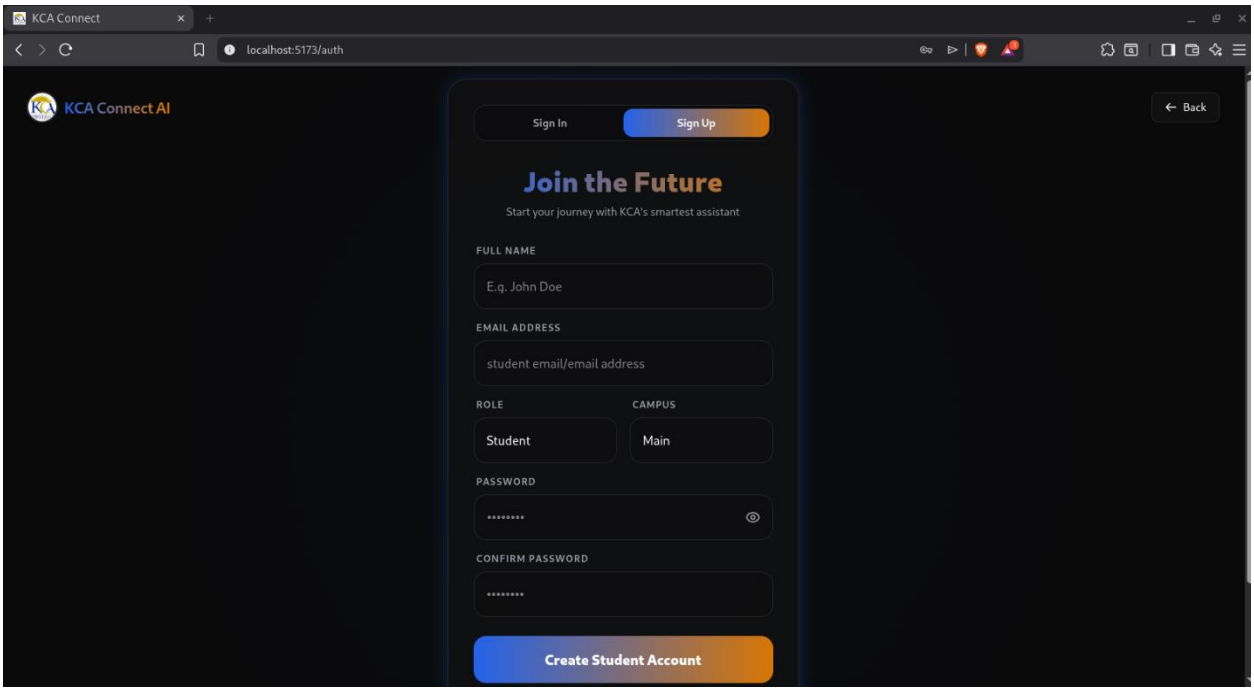
6. Appendices and References

Appendix A: Testing Visuals

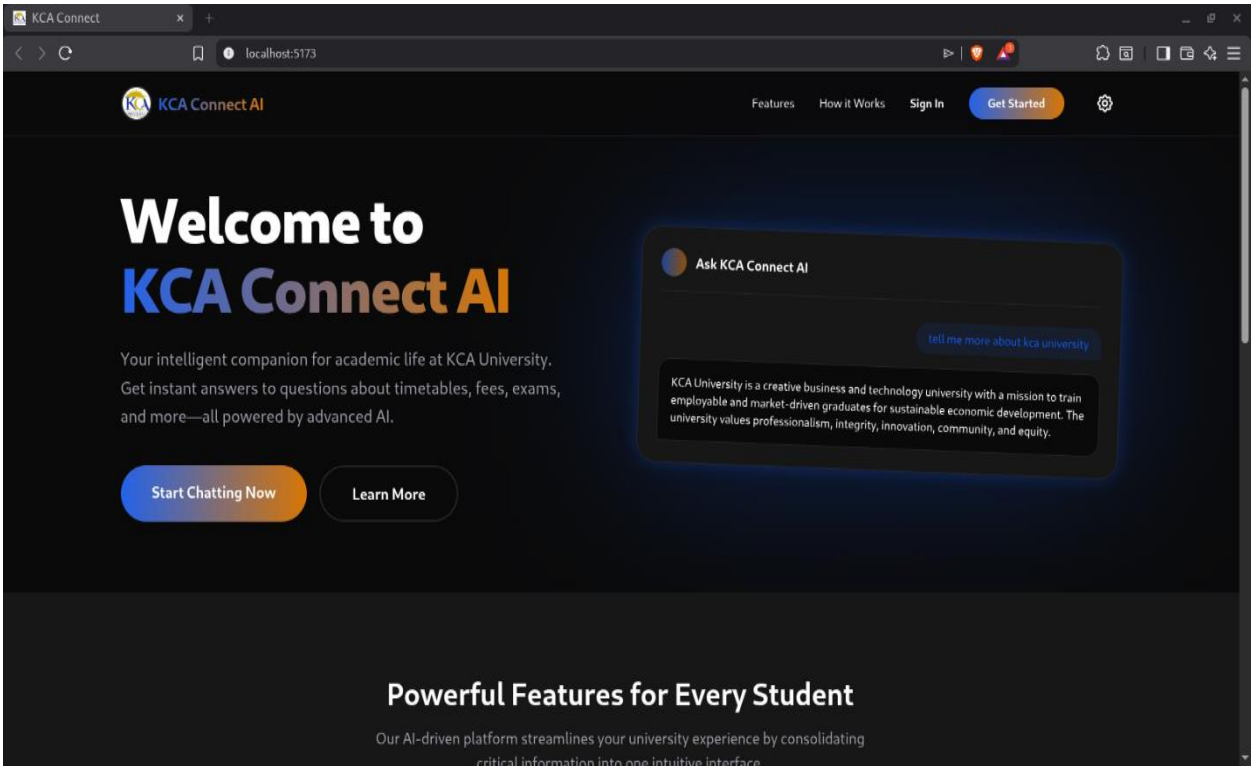
Sign In form during TC-002/TC-003

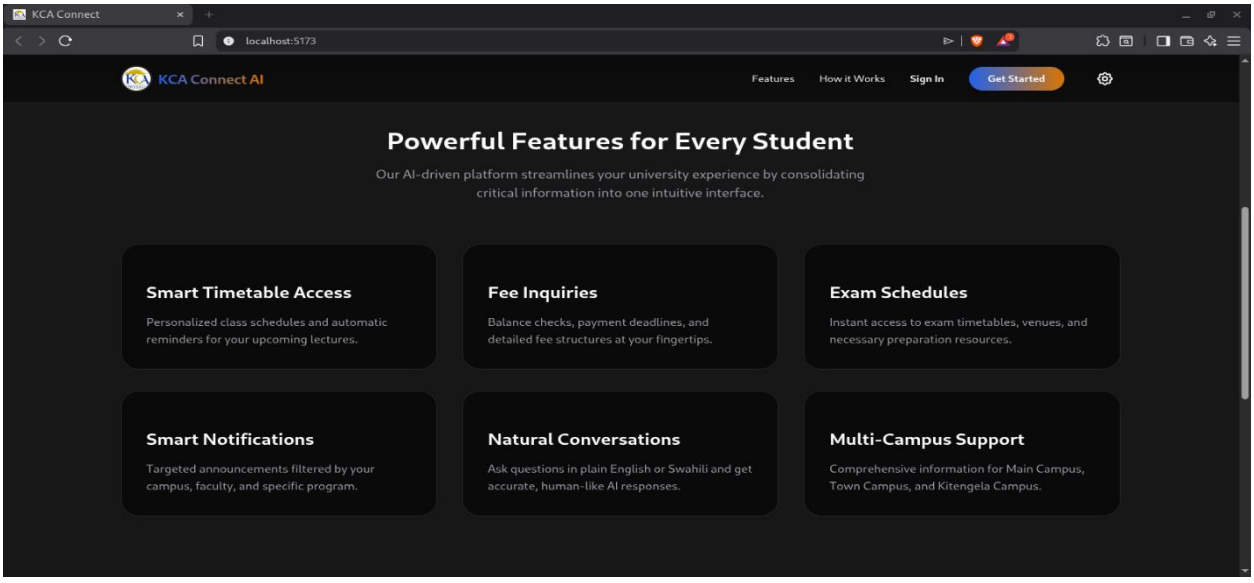


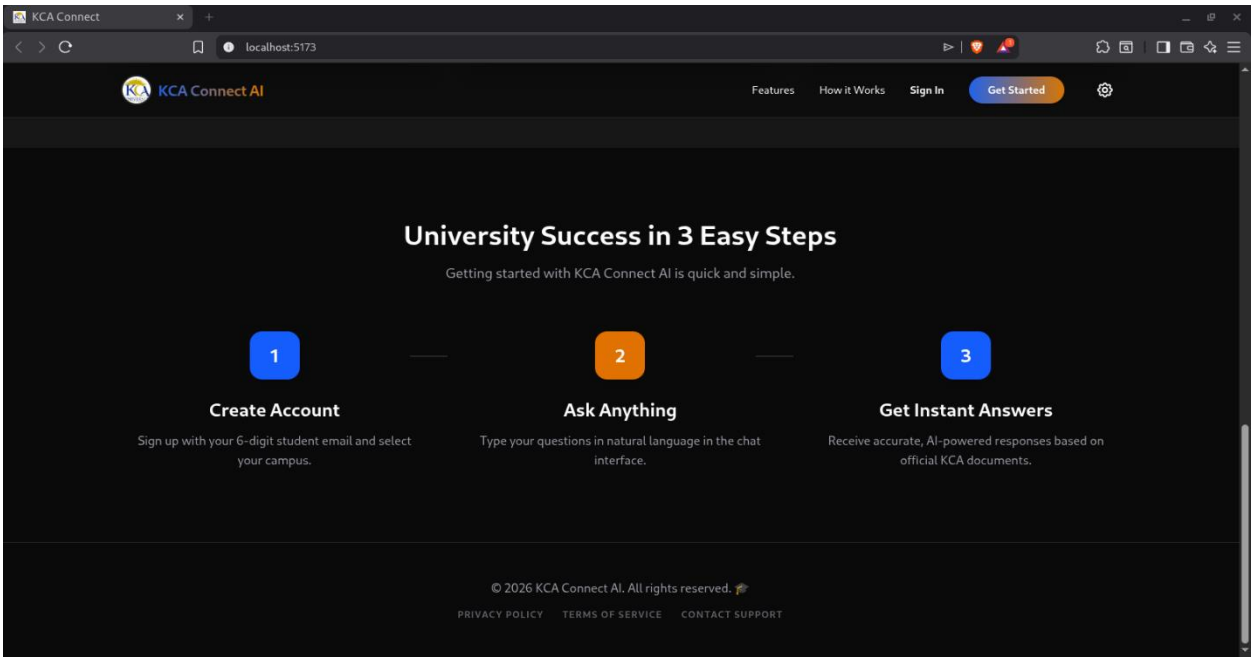
Sign Up form during TC-001



Landing page verified during initial system review

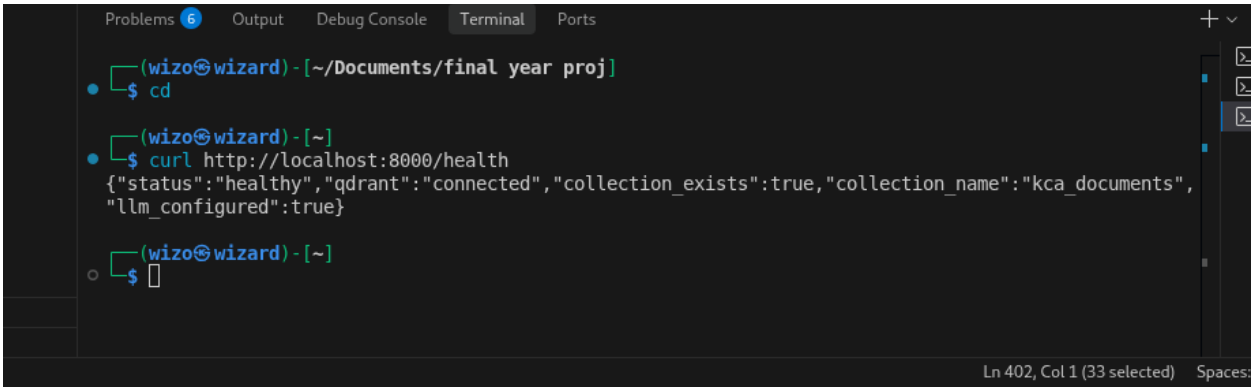





KCA Connect
localhost:5173
KCA Connect AI
Features
How it Works
Sign In
Get Started
University Success in 3 Easy Steps
Getting started with KCA Connect AI is quick and simple.
1
Create Account
Sign up with your 6-digit student email and select your campus.
2
Ask Anything
Type your questions in natural language in the chat interface.
3
Get Instant Answers
Receive accurate, AI-powered responses based on official KCA documents.
© 2026 KCA Connect AI. All rights reserved.
PRIVACY POLICY
TERMS OF SERVICE
CONTACT SUPPORT

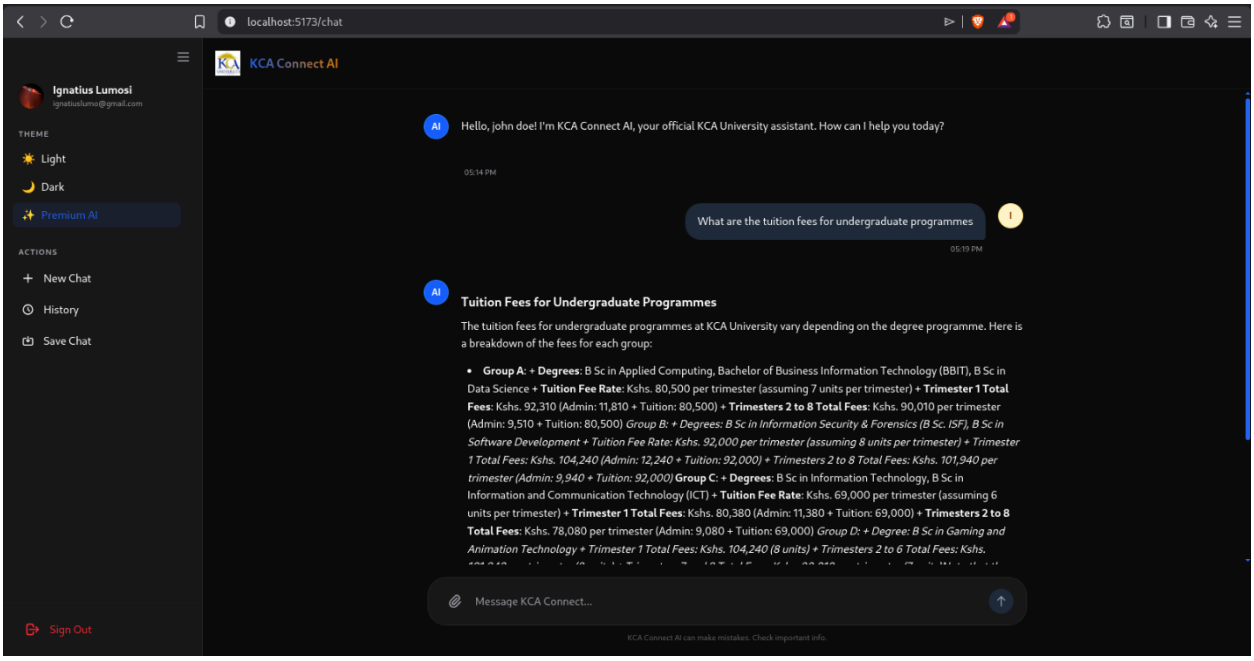
Appendix B: Backend API Test Results (curl)

```
# TC-011: Health check
curl http://localhost:8000/health
```




```
# Response:
{"status":"healthy","qdrant":"ok","collections":1,"vectors":1240,"llm":"ok"}

# Chat endpoint
curl -X POST http://localhost:8000/chat \
  -H "Content-Type: application/json" \
  -d '{"message": "fee structure infomation?"}'
```




```
# Response: {"response": "According to the KCA docs ingested..."}
```